1 Significant Digits, Accuracy, Precision, and Errors

The significant digits of a number are the digits that carry meaningful information about its value, including all certain digits and the first uncertain (estimated) digit.

For example:

- 12.34 has 4 significant digits.
- 0.01234 has 4 significant digits.
- 1200 may be ambiguous depending on the notation used.
- 1.200 has 4 significant digits, since the trailing zeros are significant.
- 0.004050 has 4 significant digits: 4, 0, 5, and 0.

An Important Point

Significant digits are not the same as decimal places.

For example,

$$0.00452$$

has 3 significant digits:

$$4, \quad 5, \quad 2,$$

but it has 5 decimal places.

From the perspective of numerical analysis, this can be interpreted as follows:

> Significant digits indicate the precision with which a numerical value is represented.

For example,

$$x = 3.142$$

has 4 significant digits. The fourth significant digit indicates the precision with which the value is represented.

The concepts of significant digits, accuracy, precision, and relative error are closely related, but they describe different aspects of numerical computation.

1.1 Significant Digits

Significant digits represent the reliable or meaningful digits of a number.

For example, suppose that the exact value is

$$x = 12.34567.$$

If we represent it by

$$\tilde{x} = 12.35,$$

then $\tilde{x}$ has 4 significant digits.

1.2 Accuracy

Accuracy describes how close an approximate value is to the true value.

For example, let

$$x = 1.234567$$

and consider the two approximations

$$x_1 = 1.2345, \qquad x_2 = 1.20.$$

Then x_1 is more accurate than x_2 because it is closer to the true value.

The absolute error is

$$E_a = |x - \tilde{x}|,$$

while the relative error is

$$E_r = \frac{|x - \tilde{x}|}{|x|}.$$

1.3 Precision

Precision describes how finely or consistently a numerical value is represented.

For example, if a device produces measurements such as

$$1.2341, \qquad 1.2342, \qquad 1.2341,$$

then the measurements are very close to each other, indicating high precision.

However, if the true value is 1.2500, these measurements may still be far from the true value. Thus, high precision does not necessarily imply high accuracy.

$$\boxed{\text{Precision} \neq \text{Accuracy}}$$

1.4 Relative Error and Significant Digits

Suppose

$$x = 12.3456$$

and the approximate value is

$$\tilde{x} = 12.35.$$

Then the absolute error is

$$|x - \tilde{x}| = |12.3456 - 12.35| = 0.0044.$$

The relative error is

$$E_r = \frac{0.0044}{12.3456} \approx 3.56 \times 10^{-4}.$$

Thus, the relative error is approximately

$$0.0356\%.$$

This tells us how accurate the approximation is relative to the size of the true value.

1.5 A Useful Comparison

Consider the two numbers

$$12.34 \qquad \text{and} \qquad 1234.$$

Both have 4 significant digits.

However, the last significant digit of 12.34 is at the 10^{-2} level, whereas the last significant digit of 1234 is at the 10^{0} level.

Thus, significant digits are used to describe how much meaningful information is contained in a number, independently of the magnitude of the number.

Significant digits
$\downarrow$
How many meaningful digits?

Accuracy
$\downarrow$
How close to the true value?

Precision
$\downarrow$
How finely/consistently represented?

Moreover,

$$\boxed{\text{Relative error} = \frac{\text{absolute error}}{\text{true value}}}$$

Significant digits are directly related to relative error. For example, if a calculation produces approximately p significant digits, the relative error is typically expected to be of order 10^{-p}, depending on the rounding convention used.

2 Chopping Error and Round-off Error

Chopping error and round-off error can be understood by approximating the same number in two different ways.

2.1 Chopping Error

Suppose

$$x = 3.14159265$$

and we want to represent it using 4 significant digits.

With chopping, we simply discard all digits after the fourth significant digit:

$$x_{\text{chop}} = 3.141.$$

The absolute chopping error is

$$E_c = |3.14159265 - 3.141| = 0.00059265.$$

2.2 Round-off Error

If we round the same number to 4 significant digits,

$$x = 3.14159265,$$

the first four significant digits are

$$3.141.$$

Since the next digit is 5, we round upward:

$$x_{\text{round}} = 3.142.$$

The round-off error is

$$E_r = |3.14159265 - 3.142| = 0.00040735.$$

Therefore,

$$\boxed{E_r < E_c}.$$

Thus, in this example, rounding produces a smaller error than chopping.

2.3 A Simpler Example

Consider

$$x = 12.6789.$$

Suppose that we want to represent the number using 4 significant digits.

With chopping,

$$12.6789 \longrightarrow 12.67,$$

and hence

$$|12.6789 - 12.67| = 0.0089.$$

With rounding, the fifth significant digit is 8, so

$$12.6789 \longrightarrow 12.68.$$

Therefore,

$$|12.6789 - 12.68| = 0.0011.$$

Hence,

$$\boxed{0.0011 < 0.0089}.$$

Thus, rounding gives a smaller error in this example.

2.4 Why Is Rounding Usually Better?

Suppose that a number is represented using n decimal places.

For chopping, the error is approximately bounded by

$$|E_c| < 10^{-n}.$$

For rounding, the error is approximately bounded by

$$|E_r| \leq \frac{1}{2}10^{-n}.$$

For example, if 3 decimal places are used, then

$$|E_c| < 0.001,$$

whereas

$$|E_r| \leq 0.0005.$$

Therefore, the maximum rounding error is approximately half the corresponding chopping error.

3 A Numerical Analysis Example

Consider

$$x = 0.123456789$$

and suppose that we use 5 significant digits.

With chopping,

$$x_{\text{chop}} = 0.12345.$$

With rounding,

$$x_{\text{round}} = 0.12346.$$

The rounding approximation is closer to the true value.

This illustrates an important point in numerical analysis: rounding is generally preferable because the small errors introduced at each step of a computation can accumulate.

3.1 Chopping Error in Scientific Notation

Consider

$$x = 123.456789.$$

Its normalized representation is

$$x = 1.23456789 \times 10^2.$$

Using 5 significant digits and chopping the mantissa gives

$$1.2345 \times 10^2.$$

Thus,

$$x_c = 1.2345 \times 10^2 = 123.45.$$

The corresponding error is

$$E_c = |123.456789 - 123.45| = 0.006789.$$

3.2 Round-off Error

For the same number,

$$x = 1.23456789 \times 10^2,$$

the first 5 significant digits are

$$1.2345.$$

Since the next digit is 6, we round upward:

$$1.2345 \longrightarrow 1.2346.$$

Therefore,

$$x_r = 1.2346 \times 10^2 = 123.46.$$

The round-off error is

$$E_r = |123.456789 - 123.46| = 0.003211.$$

Consequently,

$$\boxed{E_r < E_c}.$$

4 A More Illustrative Example

Consider

$$x = 9.87654321 \times 10^3.$$

Suppose that we use 4 significant digits.

4.1 Chopping

Keeping only the first four significant digits of the mantissa gives

$$x_c = 9.876 \times 10^3 = 9876.$$

Hence,

$$|9876.54321 - 9876| = 0.54321.$$

4.2 Rounding

The first four significant digits are

$$9.876.$$

Since the next digit is 5, we round upward:

$$9.876 \longrightarrow 9.877.$$

Thus,

$$x_r = 9.877 \times 10^3 = 9877.$$

The corresponding error is

$$|9876.54321 - 9877| = 0.45679.$$

Therefore,

$$\boxed{\text{rounding error} < \text{chopping error}}.$$

5 General Representation

Suppose that

$$x = m \times 10^e, \qquad 1 \leq m < 10,$$

and suppose that the mantissa contains p significant digits.

We denote the chopped value by

$$\boxed{x_c = \text{chop}(x)}$$

and the rounded value by

$$\boxed{x_r = \text{round}(x)}.$$

In a normalized decimal floating-point system, rounding is performed by examining the first discarded digit. If that digit is 5 or larger, the last retained digit is increased by one.

This notation is useful when introducing machine arithmetic and machine epsilon. In particular, we can write the floating-point representation in the form

$$fl(x) = x(1 + \delta),$$

where δ represents the relative error introduced by floating-point representation.

6 Floating-Point Arithmetic

For simplicity, suppose that a computer stores only 4 significant digits.

We write a normalized floating-point number as

$$fl(x) = m \times 10^e, \qquad 1 \le m < 10.$$

We now consider several examples.

6.1 Example 1: Addition

Suppose

$$x = 1.234, \qquad y = 5.678.$$

The exact sum is

$$x + y = 6.912.$$

Since the result already contains 4 significant digits,

$$fl(x + y) = 6.912.$$

Now consider

$$x = 1.234, \qquad y = 5.6789.$$

First, represent y in the computer's floating-point format.

Chopping

$$5.6789 \longrightarrow 5.678.$$

Therefore, the computer evaluates

$$fl(x) + fl(y) = 1.234 + 5.678 = 6.912.$$

However, the exact result is

$$1.234 + 5.6789 = 6.9129.$$

After chopping to 4 significant digits,

$$fl(6.9129) = 6.912.$$

Thus, the computer result is

$$\boxed{6.912},$$

while the exact result is

$$6.9129.$$

The resulting error is

$$0.0009.$$

Rounding

Now round 5.6789 to 4 significant digits:

$$5.6789 \longrightarrow 5.679.$$

Then

$$1.234 + 5.679 = 6.913.$$

The exact result remains

$$6.9129.$$

Thus, the computer gives

$$6.913,$$

with error

$$|6.9129 - 6.913| = 0.0001.$$

This example clearly illustrates that rounding can produce a smaller error than chopping.

7 Example 2: Multiplication

Consider

$$x = 2.345, \qquad y = 1.234.$$

The exact product is

$$2.345 \times 1.234 = 2.894330.$$

With 4 significant digits, chopping gives

$$2.894330 \longrightarrow 2.894.$$

The error is

$$|2.894330 - 2.894| = 0.000330.$$

For rounding, the next digit is 3, so no increase occurs:

$$2.894330 \longrightarrow 2.894.$$

Thus, in this example, chopping and rounding give the same result.
This shows an important point:

> Rounding does not always produce a different result from chopping.

The difference appears when the first discarded digit causes the last retained digit to change.

8 A More Illustrative Example: Successive Operations

Consider

$$a = 1.234, \qquad b = 5.678.$$

Suppose we want to evaluate

$$(a + b) - b.$$

Mathematically,

$$(a + b) - b = a = 1.234.$$

Using four significant digits, the computer first computes

$$a + b = 1.234 + 5.678 = 6.912.$$

Then

$$6.912 - 5.678 = 1.234 = a$$

In this case there is no problem.
Now consider

$$a = 1.000, \qquad b = 0.00049.$$

Mathematically,

$$(a + b) - a = b = 0.00049.$$

However, using four significant digits, the computer first computes

$$a + b = 1.00049.$$

After chopping,

$$1.00049 \longrightarrow 1.000.$$

Therefore,

$$(1.000 + 0.00049) - 1.000$$

is computed by the computer as

$$1.000 - 1.000 = 0.$$

The exact answer, however, is

$$0.00049.$$

Thus, a small number can lose its significance when it is added to a much larger number.
This is an important motivation for studying floating-point arithmetic.

9 Floating-Point Approximation

In a computer, the result of an exact operation is generally converted into a floating-point approximation:

$$\text{exact result} \quad \longrightarrow \quad \text{floating-point approximation.}$$

During this conversion, chopping or rounding error may occur.
Consequently, instead of interpreting a computer operation simply as

$$x + y,$$

it is more accurate to write

$$fl(x + y).$$

After several operations, we may obtain expressions such as

$$fl\big(fl(x) + fl(y)\big).$$

This is the foundation of floating-point arithmetic.

10 A Simple Binary Example

Suppose that a computer uses a binary floating-point system with 4 significant binary digits.

Consider

$$x = (101.1011)_2.$$

In normalized form,

$$x = (1.011011)_2 \times 2^2.$$

Keeping the first four significant binary digits gives

$$1.011011 \longrightarrow 1.011.$$

Thus, with chopping,

$$x_c = (1.011)_2 \times 2^2.$$

Since

$$(1.011)_2 = 1 + \frac{1}{2} + \frac{1}{8} = 1.375,$$

we obtain

$$x_c = 1.375 \times 4 = 5.5.$$

The exact value is

$$(101.1011)_2 = 5.6875.$$

Hence,

$$E_c = |5.6875 - 5.5| = 0.1875.$$

10.1 Binary Rounding

Consider the same number:

$$x = (1.011011)_2 \times 2^2.$$

The first four significant binary digits are

$$1.011.$$

The next bit is 0, so no rounding is necessary:

$$x_r = (1.011)_2 \times 2^2 = 5.5.$$

Thus, in this example, chopping and rounding give the same result.

11 A Binary Example Showing the Difference

Now consider

$$x = (101.1111)_2.$$

In normalized form,

$$x = (1.011111)_2 \times 2^2.$$

Keeping four significant binary digits gives

$$1.011.$$

The next bit is 1, so rounding gives

$$1.011 \longrightarrow 1.100.$$

Thus, with chopping,

$$x_c = (1.011)_2 \times 2^2 = (101.1)_2 = 5.5.$$

With rounding,

$$x_r = (1.100)_2 \times 2^2 = (110)_2 = 6.$$

The exact value is

$$(101.1111)_2 = 5.9375.$$

Therefore,

$$E_{\text{chop}} = |5.9375 - 5.5| = 0.4375,$$

while

$$E_{\text{round}} = |5.9375 - 6| = 0.0625.$$

Hence,

$$\boxed{E_{\text{round}} = 0.0625 \ll E_{\text{chop}} = 0.4375}.$$

This example clearly demonstrates why rounding is generally preferred in floating-point arithmetic.